

Authentication Guide

Project **multi-tenant-ecommerce**: NestJS + Prisma backend, **platform-admin** frontend on React + Redux

A step-by-step walkthrough: how authentication works in the project, what is implemented where on the backend and frontend, which concepts to know, what to remember and what to watch out for.

Repository	github.com/ShipaM/multi-tenant-ecommerce
Code state	commit ee4653a from 2026-09-29 (default branch)
Covered	backend/src/auth , backend/src/users , backend/src/common , backend/prisma , platform-admin/src (store, lib, api, routes, pages)
Audience	Team members who work on or integrate with authentication

REMEMBER How to read this guide

Parts 1-4 give the overview and glossary. Parts 5-6 cover the backend and frontend layer by layer. Part 7 gives step-by-step scenarios (login, 2FA, refresh, logout, password change and reset). Parts 8-9 cover strengths and known gaps. Part 10 is a checklist and review questions.

Boxes: **REMEMBER** = key point, **WARNING** = pitfall, **IDEA** = improvement. File paths are given from the repository root.

Contents

1. The Big Picture	4
1.1. What the project consists of	4
1.2. Overall diagram	4
1.3. What authentication does in the project	4
2. Key Concepts	5
3. Database (Prisma)	7
3.1. Tables and fields worth understanding	7
3.2. Migration history	8
4. Four Kinds of JWT	8
4.1. Why two tokens (access + refresh)	8
4.2. Login timeline	9
5. Backend by layers (NestJS)	9
5.1. The request path through the server	9
5.2. main.ts: global settings	9
5.3. app.module.ts, configuration and .env	10
5.4. Authentication endpoints	11
5.5. AuthService: what each method does	12
5.6. How route protection works: Guard + Strategy + Decorator	12
5.7. UsersService	13
5.8. DTOs and input validation	13
5.9. Other pieces: User-Agent, email, error filter	13
6. Frontend by layers (platform-admin)	14
6.1. Frontend file map	14
6.2. Token storage: lib/storage.ts	14
6.3. lib/axios.ts: the heart of frontend authentication	15
6.4. Redux: authSlice	15
6.5. Routes and page protection	16
6.6. The authentication screens and how they communicate	16
7. Step-by-step flows	17
7.1. Login without 2FA	17
7.2. Login with two-factor authentication	18
7.3. Request to a protected endpoint	19
7.4. Automatic token refresh	19
7.5. Logout and session management	20
7.6. Enabling and disabling 2FA	20
7.7. Changing the password (current one is known)	21
7.8. Password recovery (forgotten password)	21

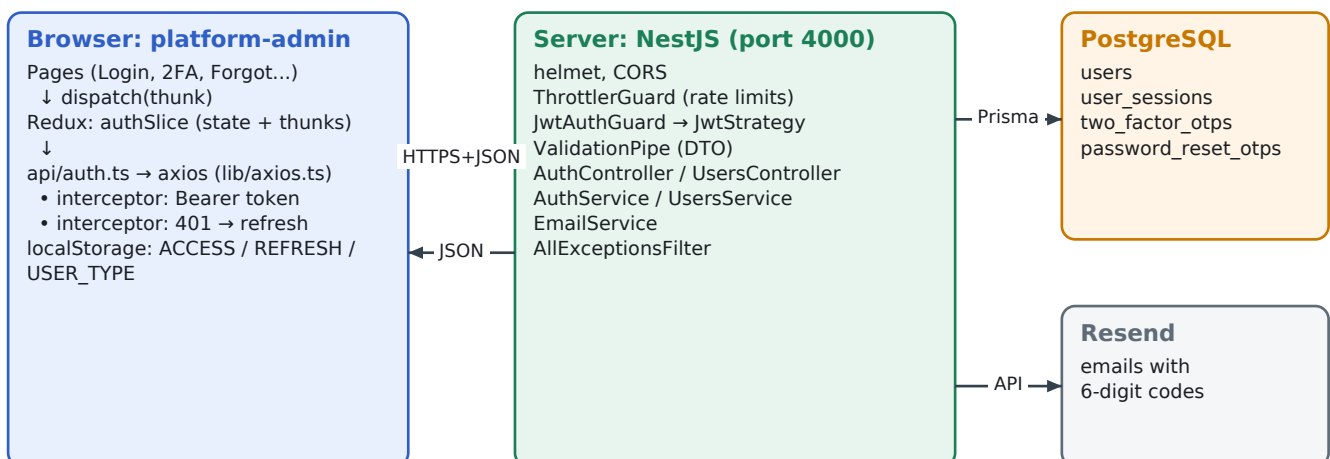
- 7.9. Editing the profile 22
- 8. Strengths of the current design 22**
- 9. Known gaps and recommendations 23**
- 9.1. What true multi-tenancy requires 25
- 10. Checklist and review questions 25**
- 10.1. Checklist: 15 key points 25
- 10.2. Onboarding and review questions 25
- Appendices 27**
- A. How to observe the flows 27
- B. Authentication file map 28
- C. For more detail 28

1. The Big Picture

1.1. What the project consists of

Part	Technologies	Role in authentication
backend/ (port 4000)	NestJS 12, Prisma 7 + PostgreSQL, passport-jwt, @nestjs/jwt, bcryptjs, @nestjs/throttler, helmet, class-validator, Resend (email), ua-parser-js, vitest	Verifies the password, issues and verifies tokens, stores sessions and one-time codes, sends emails with codes
platform-admin/ (port 3002)	React 19, Vite, react-router, Redux Toolkit, axios, shadcn/ui, sonner, moment	Login forms, token storage, automatic token refresh, page protection, the "My account" page
PostgreSQL	tables users , user_sessions , two_factor_otps , password_reset_otps	The single source of truth about users and sessions
Resend	external email service	Delivers 6-digit codes (2FA and password reset)

1.2. Overall diagram



1.3. What authentication does in the project

- **Login by email + password** (**POST** `/auth/login`) issuing a token pair and creating a session.
- **Two-factor authentication (2FA)** with a code from an email: both at login and as a setting that can be enabled/disabled.
- **Access + refresh tokens** with automatic renewal on the frontend and refresh token **rotation**.
- **Server-side sessions** (`user_sessions` table): device list, logout of a specific device, logout of all other devices.
- **Logout** (**POST** `/auth/logout`) with immediate token invalidation.
- **Password change** (current password known) and **password recovery** (forgotten) via a code from an email.
- **Profile** (**PUT** `/users/me`): name, email, phone, photo.
- **Brute-force protection**: request rate limits, a limit on code entry attempts, identical responses for non-existent emails.

WARNING About "multi-tenant": a clarification

The project name says multi-tenant, but the **authentication code has no concept of a "tenant" (store/organization) yet**: the DB schema has no tenants table and tokens carry no `tenantId`. There is only the `userId` field (`CUSTOMER`, `SELLER`, `PLATFORM_ADMIN`, `DELIVERY_AGENT`).

The `platform-admin` frontend is the platform owner's console (text on the login screen: "Run every store on your marketplace from one place"). Data isolation between tenants is the next stage; Part 9 notes what needs to be added for it.

2. Key Concepts

This is a glossary to come back to. The "Where in the project" column shows where each concept is used.

Basics

Concept	What it is	Where in the project
Authentication (authN)	Determine who is making the request: check the email and password, the code from the email.	<code>AuthService.login</code> , <code>JwtStrategy</code>
Authorization (authZ)	Decide what this user is allowed to do : roles, permissions, access to tenant data.	Not implemented in the project yet (see Part 9)
Session	A record saying "this user is logged in on this device". Makes it possible to revoke a login.	<code>user_sessions</code> table
Stateless / stateful	Stateless: the server stores nothing and trusts the token signature. Stateful: the server stores state (a session) and checks against it.	Hybrid: JWT + a session check in the DB on every request

Tokens

Concept	What it is	Where in the project
JWT	A string <code>header.payload.signature</code> . The payload is plain base64url (anyone can read it), the signature prevents forging it.	<code>JwtService.signAsync</code> / <code>verifyAsync</code>
Access token	Short-lived (15 min) token for API requests in the <code>Authorization: Bearer ...</code> header.	<code>signTokenPair</code> , <code>JwtStrategy</code>
Refresh token	Long-lived (30 days) token, used only to obtain a new token pair.	<code>AuthService.refresh</code> , <code>lib/axios.ts</code>
Refresh rotation	Each renewal issues a new refresh token and the old one stops working.	<code>refresh()</code> → <code>userSession.update</code>
Bearer	Token transport scheme: <code>Authorization: Bearer <token></code> .	interceptor in <code>lib/axios.ts</code>
Signing secret	A string of at least 32 characters used to sign JWTs. Each token type has its own.	<code>JWT_*_SECRET</code> in <code>.env</code>

Passwords, codes and hashes

Concept	What it is	Where in the project
Hash	An irreversible "fingerprint" of data. The password is stored in the DB only as a hash.	<code>passwordHash</code>
bcrypt	A slow, salted password hash; the <code>cost=10</code> parameter sets how slow it is.	<code>bcrypt.hash(pw, 10)</code>
HMAC-SHA256	A hash with a secret key. It hashes the refresh token before it is written to the DB.	<code>hashRefreshToken</code>
OTP	One-Time Password: a single-use 6-digit code, valid for 5 minutes, allows 5 attempts.	<code>two_factor_otps</code> , <code>password_reset_otps</code>
2FA	Second factor: besides knowing the password, the user must prove access to the mailbox.	<code>twoFactorEnabled</code>
Timing attack	Guessing from response time. Defense: constant-time comparison.	<code>timingSafeEqual</code> , dummy hash
Email enumeration	Finding out whether an email is registered from differences in responses.	identical responses in <code>login</code> , <code>forgotPassword</code>

Backend (NestJS)

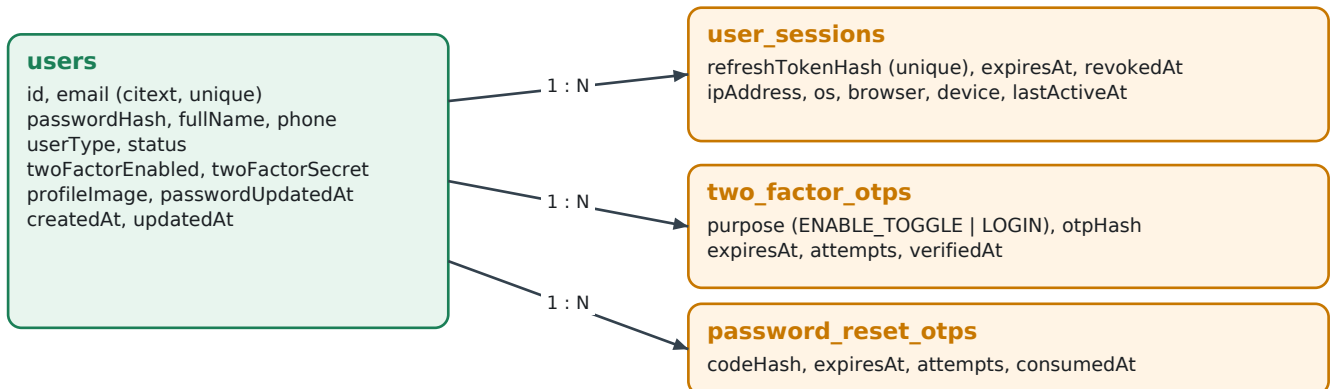
Concept	What it is	Where in the project
Module / Controller / Service	A module groups code; a controller receives the HTTP request; a service holds the logic.	<code>AuthModule</code> , <code>AuthController</code> , <code>AuthService</code>
DTO + ValidationPipe	A DTO is a class describing the request body; a pipe validates and cleans the data before the controller.	<code>LoginDto</code> , <code>main.ts</code>
Guard	Decides whether a request may reach the handler. Runs before pipes.	<code>JwtAuthGuard</code> , <code>ThrottlerGuard</code>
Passport Strategy	A rule for verifying the request credentials (here, the JWT from the header).	<code>JwtStrategy</code>
Decorator	<code>@Post</code> , <code>@UseGuards</code> , <code>@Throttle</code> , <code>@CurrentUser</code> : annotations/tools applied to methods and parameters.	<code>current-user.decorator.ts</code>
Exception Filter	A single place where errors are turned into HTTP responses.	<code>AllExceptionsFilter</code>
Prisma	ORM: the schema is described in <code>schema.prisma</code> , producing a typed client.	<code>PrismaService</code>

Frontend (React)

Concept	What it is	Where in the project
Redux Toolkit slice	State store + reducers for one area (auth).	<code>store/auth/authSlice.ts</code>
<code>createAsyncThunk</code>	An async action with pending / fulfilled / rejected states.	<code>fetchLogin</code> , <code>fetchMe...</code>
<code>unwrap()</code>	Turns the thunk result into a plain promise: success → value, error → <code>throw</code> .	<code>dispatch(...).unwrap()</code> in pages
Interceptor (axios)	Intercepts every request/response: adds the token, catches 401.	<code>lib/axios.ts</code>
Protected route	A wrapper route that blocks access without a token.	<code>ProtectedRoute.tsx</code>
<code>localStorage</code>	Browser storage; survives reloads and is readable by any JS on the page.	<code>lib/storage.ts</code>
Open redirect	An attack via a parameter like <code>?redirect_uri=https://evil.com</code> .	<code>isSafeRedirectPath</code>

3. Database (Prisma)

File: `backend/prisma/schema.prisma`. All related tables reference `users` and have `onDelete: Cascade`: when a user is deleted, their sessions and codes are deleted automatically.



3.1. Tables and fields worth understanding

Table.field	Meaning
<code>users.email</code>	Type <code>ciphertext</code> (PostgreSQL extension): case-insensitive comparison, plus <code>@unique</code> . <code>Admin@x.com</code> and <code>admin@x.com</code> are the same user.
<code>users.passwordHash</code>	bcrypt hash of the password. Never returned to the client (<code>omit</code> is used in responses).
<code>users.userType</code>	User type (enum). Currently only stored and put into the token ; access is not checked against it.
<code>users.status</code>	ACTIVE / SUSPENDED / INACTIVE . Only ACTIVE users can log in and use the API (checked at login, at refresh and on every request).
<code>users.twoFactorEnabled</code>	Whether 2FA is enabled. If yes, a code from an email is required after the password.
<code>users.twoFactorSecret</code>	Reserved for TOTP apps (Google Authenticator). Currently unused : codes are sent by email.
<code>users.passwordUpdatedAt</code>	When the password was last changed (shown on the account page: "changed ... ago").
<code>user_sessions</code> (row)	One login on one device. Its <code>id</code> goes into the access token as <code>sid</code> .
<code>user_sessions.refreshTokenHash</code>	HMAC hash of the current refresh token (the token itself is not stored).
<code>user_sessions.expiresAt</code>	Session lifetime = the refresh token <code>exp</code> . Updated on every refresh.
<code>user_sessions.revokedAt</code>	If not <code>null</code> , the session is revoked and its tokens stop working immediately.
<code>user_sessions.os / browser / device / ipAddress</code>	Parsed User-Agent (ua-parser-js) and IP, used for the device list.
<code>two_factor_otps.purpose</code>	LOGIN (code for login) or ENABLE_TOGGLE (code for enabling/disabling 2FA). A code for one purpose does not work for the other.
<code>*_otps.otpHash / codeHash</code>	bcrypt hash of the 6-digit code: the code itself is not in the DB.
<code>*_otps.attempts</code>	Counter of wrong attempts. After 5: "Too many otp attempts".
<code>two_factor_otps.verifiedAt, password_reset_otps.consumedAt</code>	When the code was successfully used: makes the code single-use.

3.2. Migration history

Migration	What it added
20260907 <code>users_session_tables</code>	<code>citext</code> , <code>enums</code> , <code>users</code> , <code>user_sessions</code> , <code>password_reset_otps</code>
20260910 <code>profile_image</code>	<code>users.profile_image</code>
20260917 <code>two_factor_otp_and_expiry_indexes</code>	<code>two_factor_otps</code> , indexes on <code>expires_at</code>
20260918 <code>two_factor_otp_purpose</code>	<code>purpose</code> field (LOGIN / ENABLE_TOGGLE)
20260922 <code>user_agent</code>	<code>os</code> , <code>device</code> , <code>browser</code> in the session
20260924 <code>added_password_updated_at_field</code>	<code>users.password_updated_at</code>
20260929 <code>password_reset_otp_attempts</code>	<code>attempts</code> in <code>password_reset_otps</code>

REMEMBER The main idea of the schema

User $\neq$ session $\neq$ token. **A token is a "pass", a session is an "entry in the pass log"**. The pass is valid only while the log entry is neither revoked nor expired, so logout and blocking take effect immediately, not after 15 minutes.

4. Four Kinds of JWT

The project signs **four different tokens**, each with its own secret and lifetime. Separate secrets ensure a token of one kind cannot be passed off as another.

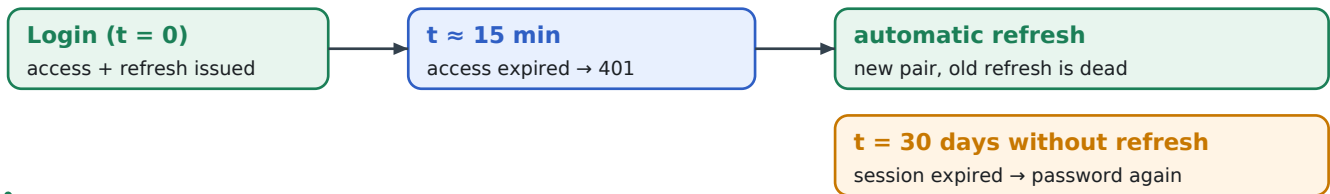
Token	Secret (.env)	TTL	Payload	Where stored	Purpose
Access	<code>JWT_ACCESS_SECRET</code>	15m	<code>userId</code> , <code>email</code> , <code>userId</code> , <code>email</code> , <code>sessionType</code> , <code>sid</code>	Client: <code>localStorage</code> . Server: nowhere	Access to protected endpoints (Bearer)
Refresh	<code>JWT_REFRESH_SECRET</code>	30d	<code>userId</code> , <code>sessionId</code>	Client: <code>localStorage</code> . Server: only the HMAC hash in <code>user_sessions</code>	Obtain a new token pair
2FA	<code>JWT_2FA_SECRET</code>	15m	<code>userId</code>	Client: router <code>location.state</code> . Server: nowhere	twoFactorToken : proves "the password is already verified" while waiting for the emailed code
Reset	<code>JWT_RESET_SECRET</code>	5m	<code>userId</code>	Client: <code>location.state</code> . Server: nowhere	resetToken : the right to set a new password after entering the code

TTL is the default lifetime. Values can be changed in `.env` (format `15m`, `7d`, `3600s`, validated by a regular expression in `env.validation.ts`).

4.1. Why two tokens (access + refresh)

- The access token is sent with **every** request, so it is more likely to leak. That is why it lives 15 minutes: a stolen token quickly becomes useless.
- The refresh token is used **rarely** (about once per 15 minutes and only on `/auth/refresh`). It lives long so users do not have to enter their password every 15 minutes.
- On the server the refresh token is bound to a session: it can be revoked (`revokedAt`) and is replaced on every use.

4.2. Login timeline



REMEMBER A JWT is signed, not encrypted

Anyone can decode a token payload (it is just base64url). The signature protects against **forgery**, not against **reading**. So passwords and secrets must never go into a token. The access token here contains `userId`, `email`, `userType` and `sid`, which is fine.

Try it: open DevTools → Application → Local Storage, copy `ACCESS_TOKEN` and run in the console `JSON.parse(atob(token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/')))` to see the payload and the `exp` field.

5. Backend by layers (NestJS)

5.1. The request path through the server

Order matters: if a request is rejected at an early step, it never reaches the later ones.

1. **helmet** (`main.ts`) — adds protective HTTP headers to every response.
2. **CORS** (`main.ts`) — allows requests only from origins listed in `CORS_ORIGINS` (for example `http://localhost:3002`). If the variable is not set, CORS is disabled.
3. **ThrottlerGuard** (global, `app.module.ts`) — request rate limit: 60 per minute by default; sensitive routes have their own, stricter limits (`@Throttle`).
4. **JwtAuthGuard** (only on protected routes via `@UseGuards`) — checks the token and the session.
5. **ValidationPipe** (global) — validates the request body against the DTO.
6. **Controller → Service → Prisma** — business logic and the database.
7. **AllExceptionsFilter** — turns any error into a single JSON shape `{ statusCode, message, error, path, timestamp }`; unhandled errors are returned as "Internal server error" (without a stack trace), while the stack is written to the log.

5.2. `main.ts`: global settings

```

app.use(helmet());
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,           // strip fields that are not in the DTO
  forbidNonWhitelisted: true, // and if extra fields are present, return 400
  transform: true,           // turn JSON into an instance of the DTO class
  transformOptions: { enableImplicitConversion: true },
}));
app.useGlobalFilters(new AllExceptionsFilter());
if (corsOrigins) app.enableCors({ origin: [...], credentials: true });
  
```

REMEMBER `whitelist + forbidNonWhitelisted` = protection against mass assignment

In `UsersService.update`, Prisma receives `data: { ...dto }`. This is safe only because the pipe does not let extra fields into the DTO. Otherwise a client could send `{ "userType": "PLATFORM_ADMIN" }` and escalate its own privileges.

5.3. `app.module.ts`, configuration and `.env`

- `ConfigModule.forRoot({ isGlobal, cache, validate: validateEnv })` — environment variables are validated **at startup**: if a secret is missing or shorter than 32 characters, the application will not start (`env.validation.ts`).
- `ThrottlerModule.forRoot + APP_GUARD: ThrottlerGuard` — a global limit of 60 requests/min.
- `GlobalPassportModule` — registers Passport with `jwt` as the default strategy.

Variable	Required?	Value
<code>DATABASE_URL</code>	yes	PostgreSQL connection string
<code>JWT_ACCESS_SECRET</code> , <code>JWT_REFRESH_SECRET</code> , <code>JWT_2FA_SECRET</code> , <code>JWT_RESET_SECRET</code>	yes	four different random strings, each at least 32 characters
<code>JWT_ACCESS_EXPIRES_IN</code> / <code>JWT_REFRESH_EXPIRES_IN</code> / <code>JWT_2FA_EXPIRES_IN</code> / <code>JWT_RESET_EXPIRES_IN</code>	no	default <code>15m</code> / <code>30d</code> / <code>15m</code> / <code>5m</code>
<code>RESEND_API_KEY</code> , <code>RESEND_FROM_EMAIL</code>	yes	API key and sender address for emails
<code>CORS_ORIGINS</code>	no	comma-separated list of origins; without it CORS is disabled
<code>PORT</code> , <code>NODE_ENV</code>	no	default <code>4000</code> and <code>development</code>

5.4. Authentication endpoints

Method and path	Protection	Limit	Body	What it does
POST /auth/login	public	10	email, password	Checks the password. Returns tokens or a 2FA requirement
POST /auth/refresh	public	20	refreshToken	Rotation: a new token pair
POST /auth/logout	Bearer	60	—	Revokes the current session
GET /auth/me	Bearer	60	—	Profile without passwordHash and twoFactorSecret
POST /auth/2fa-generate-otp	Bearer	5	—	Sends a code for enabling/disabling 2FA
POST /auth/2fa-verify-enable	Bearer	5	otp	Verifies the code and toggles twoFactorEnabled
POST /auth/2fa-verify-login-otp	public	5	otp, twoFactorToken	Completes a 2FA login (identity is proven by twoFactorToken), issues tokens
GET /auth/sessions	Bearer	60	—	The user's active sessions
POST /auth/sessions/:id/revoke	Bearer	60	—	Revoke a single session
POST /auth/sessions/revoke-others	Bearer	60	—	Revoke all except the current one
POST /auth/forgot-password	public	5	email	Sends a reset code by email
POST /auth/forgot-password/verify-otp	public	5	email, otp	Verifies the code → resetToken
POST /auth/reset-password	public	5	resetToken, password	Sets a new password (the right to do so comes from resetToken)
PUT /users/me	Bearer	60	fullName, email, phone, profileImage	Update the profile
PUT /users/change-password	Bearer	60	currentPassword, password	Change the password

"Limit" is requests per minute; 60 is the global default. By default Throttler counts requests per IP.

5.5. AuthService: what each method does

Method	Essence
<code>login(email, password, ctx)</code>	Looks up the user; always runs <code>bcrypt.compare</code> (for a non-existent user, against a pre-computed dummy hash); wrong credentials → 401 "Invalid email or password"; status other than ACTIVE → 403; if 2FA is enabled, returns <code>twoFactorRequired</code> instead of tokens; otherwise <code>issueTokenPair</code> .
<code>issueTokenPair(user, ctx)</code>	Creates a <code>sessionId</code> (cuid2), signs the token pair and inserts a row into <code>user_sessions</code> (refresh hash, expiry from <code>exp</code> , device, IP).
<code>signTokenPair(user, sessionId)</code>	Signs access and refresh in parallel (<code>Promise.all</code>) — each with its own secret and lifetime.
<code>refresh(token, ctx)</code>	4 checks: (1) JWT signature and expiry; (2) the session exists, is not revoked and not expired; (3) the token hash matches the hash in the DB (<code>timingSafeEqual</code>); (4) the user is ACTIVE . Then issues a new pair and overwrites the hash and expiry in the same session.
<code>logout(userId, sessionId)</code>	Checks that the session belongs to the user; sets <code>revokedAt</code> .
<code>me(userId)</code>	Returns the user with <code>omit: { passwordHash, twoFactorSecret }</code> .
<code>twoFactorEnable(userId, purpose)</code>	Generates a code via <code>randomInt(100000, 1000000)</code> , hashes it with <code>bcrypt</code> , invalidates earlier live codes for the same purpose, creates a record (TTL 5 min), sends an email.
<code>verify0tp(userId, purpose, otp)</code> (private)	Shared code check for both 2FA flows: takes the newest unused, unexpired code for the given purpose; 5 or more attempts → 400; wrong code → <code>attempts + 1</code> ; correct code → <code>verifiedAt = now</code> .
<code>twoFactorVerifyEnable</code>	Verifies an <code>ENABLE_TOGGLE</code> code and inverts <code>twoFactorEnabled</code> .
<code>twoFactorCreate0tpToken</code>	Sends a <code>LOGIN</code> code and signs the <code>twoFactorToken</code> .
<code>twoFactorVerifyLogin0tp</code>	Verifies the <code>twoFactorToken</code> , the user status and the <code>LOGIN</code> code, then <code>issueTokenPair</code> .
<code>listSessions / revokeSession / revokeOtherSessions</code>	Management of the device list (the user's own sessions only).
<code>forgotPassword / forgotPassword0tpVerification / resetPassword</code>	Three-step password recovery (section 7.8).

5.6. How route protection works: Guard + Strategy + Decorator

```
// common/guards/jwt-auth.guard.ts
export class JwtAuthGuard extends AuthGuard('jwt') {}

// auth/strategies/jwt.strategy.ts
super({ jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
  ignoreExpiration: false,
  secretOrKey: config.getOrThrow('JWT_ACCESS_SECRET') });

async validate(payload: JwtAccessPayload): Promise<AuthenticatedUser> {
  const session = await this.prisma.userSession.findUnique({
    where: { id: payload.sid }, include: { user: true } });
  if (!session || session.revokedAt || session.expiresAt <= new Date())
    throw new UnauthorizedException('Session is no longer valid');
  if (session.user.status !== UserStatus.ACTIVE)
    throw new UnauthorizedException('This account is not active');
  return { userId: session.userId, email: session.user.email,
    userType: session.user.userType, sessionId: session.id };
}

// in the controller
@UseGuards(JwtAuthGuard)
@Get('me') me(@CurrentUser() user: AuthenticatedUser) {...}
```

- `@UseGuards(JwtAuthGuard)` → Passport invokes the `jwt` strategy. It extracts the token from the header and checks the **signature and expiry**.
- If the token is valid, `validate(payload)` is called. The strategy additionally goes **to the DB** for the session. This is the bridge between a "stateless JWT" and a "managed session".
- Nest puts the object returned from `validate` into `request.user`; the `@CurrentUser()` decorator extracts it into a method parameter.
- `email` and `userType` are taken **from the DB**, not from the token — so changes to them take effect immediately.

REMEMBER The price of instant logout

Every protected request makes a DB query for the session. This is a deliberate trade-off: a pure JWT needs no DB, but cannot "kick out" a user before the token expires. The project's choice is security and controllability.

5.7. UsersService

Method	Essence
<code>findByEmail</code>	<code>findUnique</code> by email (citext → case-insensitive).
<code>update(userId, dto)</code>	Updates the profile without returning secret fields. An email uniqueness violation (Prisma <code>P2002</code>) → 409 "Email is already in use".
<code>changePassword</code>	Verifies <code>currentPassword</code> (otherwise 401), hashes the new one (bcrypt, cost 10) and in a single transaction updates the password + <code>passwordUpdatedAt</code> and revokes all other sessions.
<code>resetForgottenPassword</code>	The same, but without checking the old password; <code>currentSessionId</code> is not passed → all of the user's sessions are revoked.

5.8. DTOs and input validation

DTO	Rules
<code>LoginDto</code>	<code>email</code> — <code>@IsEmail</code> ; <code>password</code> — a non-empty string
<code>RefreshTokenDto</code>	<code>refreshToken</code> — <code>@IsJWT</code> (must look like a JWT)
<code>TwoFactorDto</code>	<code>otp</code> — a non-empty string
<code>TwoFactorVerifyLoginOtpDto</code>	<code>otp</code> , <code>twoFactorToken</code> — non-empty strings
<code>ForgotPasswordDto</code>	<code>email</code> — <code>@IsEmail</code> ; <code>otp</code> — an optional string (used for both step 1 and step 2)
<code>ResetPasswordDto</code>	<code>resetToken</code> — a string; <code>password</code> — a non-empty string (no length check, see section 9)
<code>ChangePasswordDto</code>	<code>currentPassword</code> — non-empty; <code>password</code> — 8 to 72 characters
<code>UpdateUserDto</code>	all fields optional: <code>fullName</code> ≤ 120, <code>email</code> (<code>@IsEmail</code>), <code>phone</code> ≤ 32, <code>profileImage</code>

The 72 limit for passwords is not arbitrary: bcrypt only considers the first 72 bytes of a password.

5.9. Other pieces: User-Agent, email, error filter

- `@ParseUserAgent()` (`common/decorators/user-agent.decorator.ts`) — parses the `User-Agent` header with the `ua-parser-js` library; the result (`os`, `browser`, `device`) is stored in the session. `@Ip()` provides the IP.
- `EmailService` — sends emails through Resend: `sendOtp2FAuser` and `sendOtpForgotPassword`. If sending fails — 500 "Failed to send OTP email".
- `AllExceptionsFilter` — a single error format; 4xx are logged as warn, 5xx as error with the stack trace.

6. Frontend by layers (platform-admin)

6.1. Frontend file map

File	Role
<code>src/lib/storage.ts</code>	Wrapper over <code>localStorage</code> (<code>ACCESS_TOKEN</code> , <code>REFRESH_TOKEN</code> , <code>USER_TYPE</code>) with <code>try/catch</code>
<code>src/lib/axios.ts</code>	Two axios clients, interceptors, automatic refresh
<code>src/api/auth.ts</code>	The <code>authApi</code> object: one function per endpoint
<code>src/store/auth/authSlice.ts</code>	Redux: state, thunks, reducers
<code>src/store/auth/auth.types.ts</code>	The <code>AuthState</code> type
<code>src/types/auth.ts</code> , <code>types/user.ts</code>	Request/response types, type guards <code>isCompleteLoginResponse</code> , <code>isTwoFactorRequiredResponse</code>
<code>src/routes/index.tsx</code>	Route table (<code>/auth/*</code> public, <code>/dashboard/*</code> protected)
<code>src/components/ProtectedRoute.tsx</code>	Blocks access without a token, loads the profile
<code>src/layouts/AuthLayout.tsx</code>	Wrapper for the sign-in screens; if a token exists, redirects to <code>/dashboard</code>
<code>src/pages/auth/*</code>	Login, Verify2FaOtp, ForgotPassword, VerifyForgotOtp, ResetPassword
<code>src/pages/dashboard/account/*</code>	Profile, password change, 2FA, session list
<code>src/components/AppSidebar.tsx</code>	Logout button (<code>fetchLogout</code>)
<code>src/lib/redirect.ts</code>	<code>isSafeRedirectPath</code> — protection against open redirect
<code>src/hooks/useOtpCountdown.ts</code>	5-minute timer for the password reset code

6.2. Token storage: `lib/storage.ts`

- Tokens are kept in **localStorage** under the keys `ACCESS_TOKEN` and `REFRESH_TOKEN`; `USER_TYPE` is stored there too.
- Every operation is wrapped in `try/catch` (`safeGetItem`, `safeSetItem...`): if storage is unavailable (private mode, cookies blocked), the application does not crash — the login simply will not survive a reload.
- At startup, `initialState` in `authSlice` reads the tokens from storage — which is why the user stays "signed in" after F5.

6.3. lib/axios.ts: the heart of frontend authentication

```
export const Axios = axios.create({ baseURL }); // main client
const refreshClient = axios.create({ baseURL }); // separate one – only for /auth/refresh

const PUBLIC_AUTH_PATHS = ["/auth/login", "/auth/refresh"];

Axios.interceptors.request.use((config) => {
  if (isPublicAuthPath(config.url)) return config; // login has no token yet
  const t = storage.getAccessToken();
  if (t) config.headers.Authorization = `Bearer ${t}`;
  return config;
});

let pendingRefresh: Promise<string> | null = null;
function fetchRefreshToken() { // "single-flight"
  pendingRefresh ??= requestTokenPair().finally(() => { pendingRefresh = null; });
  return pendingRefresh;
}

Axios.interceptors.response.use(r => r, async (error) => {
  const req = error.config;
  const hadAuthHeader = Boolean(req?.headers?.Authorization);
  if (error.response?.status !== 401 || !req || req._retry || !hadAuthHeader)
    return Promise.reject(error);
  req._retry = true;
  try { token = await fetchRefreshToken(); }
  catch (e) { storage.clearStorage(); window.location.href = "/auth/login"; throw e; }
  req.headers.Authorization = `Bearer ${token}`;
  return Axios(req); // retry the original request
});
```

Technique	Why
A separate <code>refreshClient</code>	The refresh request does not pass through the main client's interceptors — otherwise its own 401 would trigger refresh again (an infinite loop).
<code>_retry = true</code>	A single request is retried at most once.
<code>hadAuthHeader</code>	A 401 is handled only for requests that carried a token. A wrong password at login also yields 401, but refreshing the token makes no sense there.
<code>pendingRefresh</code> (single-flight)	If 5 requests get a 401 simultaneously, only one refresh goes to the server and the rest wait on the same Promise. This is critical: the refresh token is single-use, so a second parallel refresh with the same token would fail.
<code>window.location.href = '/auth/login'</code>	If the refresh failed — a full storage reset and a hard redirect to the sign-in page.

6.4. Redux: authSlice

- **State** (`AuthState`): `user`, `accessToken`, `refreshToken`, `userType`, `error`, `twoFactorToken`, `twoFactorRequired`, `sessions` and a set of loading flags (`isLoginLoading`, `isMeLoading`, `isLogoutLoading`, ...).
- **Thunks**: `fetchLogin`, `fetchMe`, `fetchLogout`, `updateUser`, `changePassword`, `twoFactorGenerateOtp`, `verifyEnableDisableTwoFactor`, `verify2FaLoginOtp`, `fetchSessionsList`, `revokeSession`, `revokeOtherSessions`, `fetchForgotPassword`, `fetchForgotPasswordOtpVerify`, `resetForgottenPassword`.
- A uniform pattern: `try { await authApi.x() } catch { return rejectWithValue(getAxiosErrorMessage(...)) }`. A page calls `dispatch(thunk).unwrap()` inside `try/catch` and shows the error text.
- `getAxiosErrorMessage` (`lib/api.error.ts`) extracts `message` from the backend response (a string or an array — in which case the first element is taken), otherwise substitutes a fallback text.
- Tokens are saved to `localStorage` **inside the thunk** (`storage.setTokens`) and reach Redux through the reducer.
- `fetchMe.rejected` with `unauthorized: true` (a real 401) clears the tokens; a network error or 5xx does **not** log the user out.

Type guards. The `/auth/login` response comes in two forms: "tokens" or "2FA required". `isCompleteLoginResponse` and `isTwoFactorRequiredResponse` (`types/auth.ts`) check the shape of the response at runtime and at the same time narrow the type in TypeScript.

6.5. Routes and page protection

```
<Route path="auth" element={<AuthLayout />}> // public
  login | 2fa | forgot-password | forgot-password/otp | reset-password
</Route>
<Route element={<ProtectedRoute />}> // protected
  <Route path="dashboard" element={<DashboardLayout />}> ... my-account ...
</Route>
```

Component	Logic
ProtectedRoute	No <code>accessToken</code> in Redux → <code><Navigate to="/auth/login?redirect_uri=<current path>"></code> . Token present → <code>dispatch(fetchMe())</code> ; while <code>user === null</code> shows "Loading..."; then <code><Outlet /></code> .
AuthLayout	If an <code>accessToken</code> already exists, calls <code>navigate('/dashboard')</code> — a signed-in user does not need the sign-in screen.
RootRedirect	The root <code>/</code> always leads to <code>/auth/login</code> .
isSafeRedirectPath	Allows only paths of the form <code>/something</code> : must start with <code>/</code> , but not with <code>//</code> and not with <code>/\</code> . This way <code>?redirect_uri=//evil.com</code> will not turn into a jump to a foreign site.

6.6. The authentication screens and how they communicate

Screen	What it does
LoginPage	Controlled inputs; <code>fetchLogin</code> ; a full response → <code>navigate(redirectTo)</code> ; 2FA → <code>navigate('/auth/2fa', { state })</code> .
Verify2FaOtpPage	Reads <code>twoFactorToken</code> , <code>email</code> , <code>redirectTo</code> from <code>location.state</code> ; no token → back to <code>/auth/login</code> ; 6-digit <code>InputOTP</code> ; <code>verify2FaLoginOtp</code> .
ForgotPasswordPage	<code>fetchForgotPassword</code> → <code>navigate('/auth/forgot-password/otp', { state: { email, createdAt } })</code> .
VerifyForgotOtpPage	Timer <code>useOtpCountdown(createdAt, 5)</code> ; after it expires — a "Resend Otp" button; <code>fetchForgotPasswordOtpVerify</code> → <code>navigate('/auth/reset-password', { state: { email, resetToken }, replace: true })</code> .
ResetPasswordPage	Client-side check that the passwords match; without <code>email/resetToken</code> in state — back to <code>/auth/forgot-password</code> ; <code>resetForgottenPassword</code> ; a "Password Updated" screen.
ProfileDetailsPage + components	<code>EditProfile</code> , <code>ChangePassword</code> , <code>TwoFactorAuthentication</code> (dialogs), <code>SessionsList</code> .

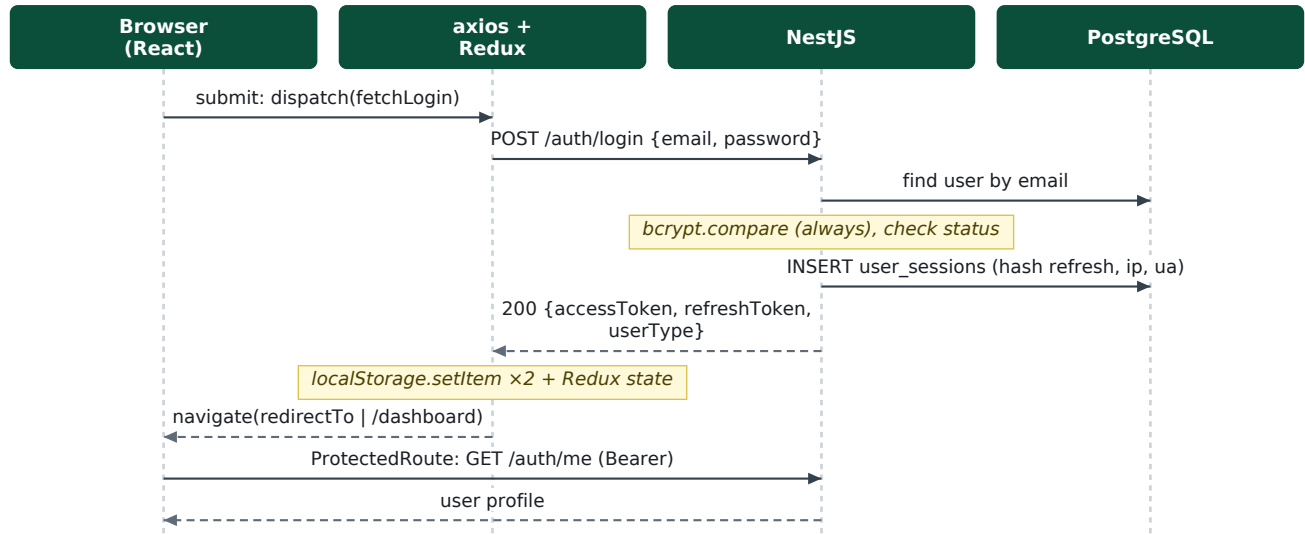
REMEMBER Intermediate tokens live in `location.state`

`twoFactorToken` and `resetToken` are passed between screens through router state (`navigate(..., { state })`), not through Redux or `localStorage`. They do not end up in the URL (they are not exposed in the address bar or in shared links). Opening `/auth/2fa` or `/auth/reset-password` directly by link is not possible: without `state` the page immediately sends the user back.

7. Step-by-step flows

This is the main part of the notes. For each flow: a diagram, then numbered steps stating **where** each action happens.

7.1. Login without 2FA

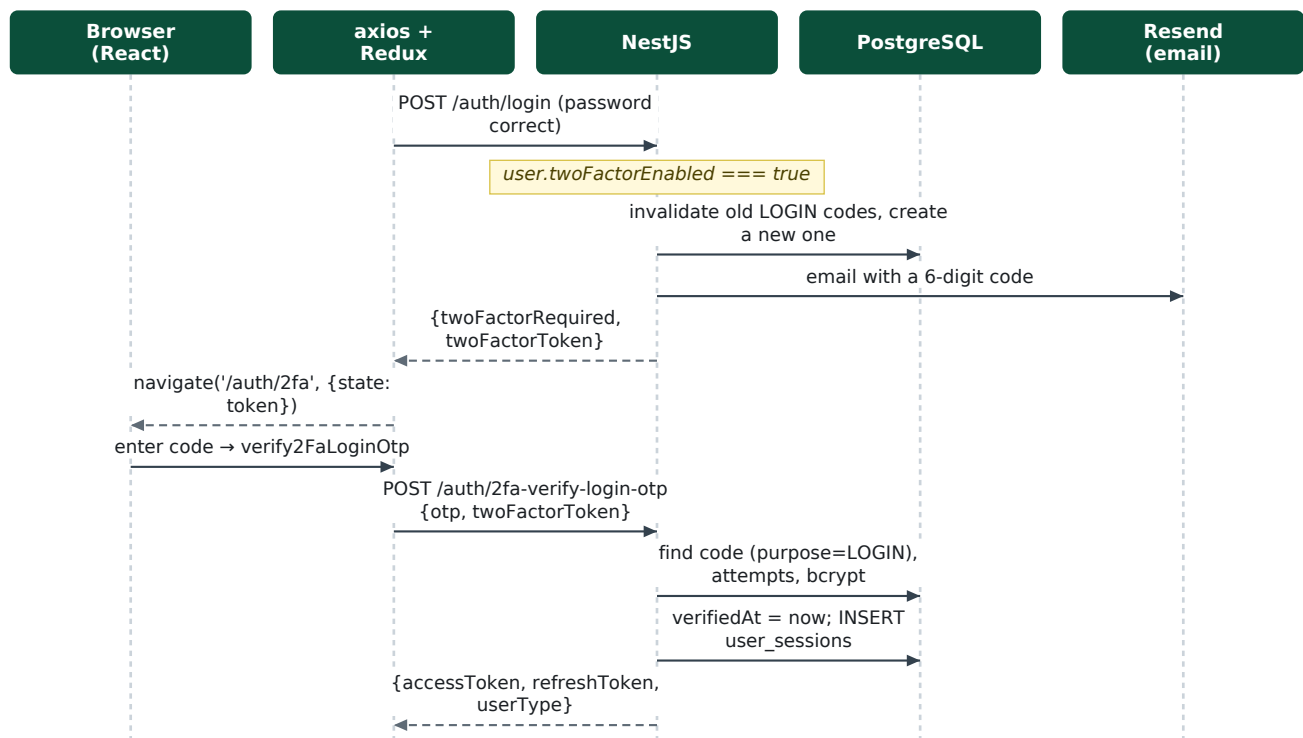


#	Where	What happens
1	Frontend	The user enters an email and password. The fields are controlled inputs (<code>useState</code>). <code>type="email"</code> and <code>required</code> are only UX conveniences, not security.
2	Frontend	Submit → <code>dispatch(fetchLogin({ email, password })).unwrap() → authApi.login → POST /auth/login</code> . The interceptor recognizes a public path and does not add <code>Authorization</code> .
3	Backend	Rate limit of 10 requests per minute; <code>ValidationPipe</code> validates <code>LoginDto</code> (email format, non-empty password).
4	Backend	<code>AuthController.login</code> collects the context: IP (<code>@Ip</code>) and the parsed <code>User-Agent</code> (<code>@ParseUserAgent</code>) → <code>{ ipAddress, device, os, browser }</code> .
5	Backend	<code>AuthService.login</code> : <code>findByEmail</code> ; then <code>bcrypt.compare(password, user?.passwordHash ?? ABSENT_USER_PASSWORD_HASH)</code> runs always , so the response time does not reveal whether the email exists.
6	Backend	No such user or wrong password → 401 "Invalid email or password" (identical in both cases). Status other than <code>ACTIVE</code> → 403 "This account is not active".
7	Backend	2FA disabled → <code>issueTokenPair</code> : a <code>sessionId</code> is created, access (15 min) and refresh (30 days) tokens are signed, the HMAC hash of the refresh token is written to <code>user_sessions</code> , and <code>expiresAt</code> is taken from its <code>exp</code> .
8	Backend	Response 200: <code>{ accessToken, refreshToken, userType }</code> .
9	Frontend	The thunk checks the response shape (<code>isCompleteLoginResponse</code>), then calls <code>storage.setTokens</code> and <code>storage.setUserType</code> . The <code>fetchLogin.fulfilled</code> reducer puts the tokens into Redux.
10	Frontend	<code>LoginPage</code> : <code>navigate(redirectTo, { replace: true })</code> . <code>redirectTo</code> is the <code>?redirect_uri</code> parameter (if it passed <code>isSafeRedirectPath</code>), otherwise <code>/dashboard</code> .
11	Frontend	<code>ProtectedRoute</code> sees a token → <code>fetchMe()</code> → <code>GET /auth/me</code> with <code>Bearer</code> . Until the profile arrives it shows "Loading...", then <code>DashboardLayout</code> is rendered.

WARNING Login failure

The thunk's `catch` calls `storage.clearStorage()` (which wipes all of `localStorage`) and `rejectWithValue(message)`. The error text is shown under the form.

7.2. Login with two-factor authentication



#	Where	What happens
1	Backend	The password is correct but <code>twoFactorEnabled = true</code> → <code>twoFactorCreateOtpToken</code> : calls <code>twoFactorEnable(userId, LOGIN)</code> . A code is generated with <code>randomInt(100000, 1000000)</code> , its bcrypt hash is written to <code>two_factor_otps</code> with a 5-minute TTL, earlier live codes for the same purpose are invalidated, and the email is sent through Resend.
2	Backend	A <code>twoFactorToken</code> is signed (JWT_2FA_SECRET, 15 min, payload { <code>userId</code> }). There are no access tokens yet. Response: { <code>twoFactorRequired: true, twoFactorToken, message</code> }.
3	Frontend	<code>isTwoFactorRequiredResponse</code> → navigation to <code>/auth/2fa</code> with state: { <code>twoFactorToken, email, redirectTo</code> }.
4	Frontend	<code>Verify2FaOtpPage</code> : the user enters 6 digits → <code>POST /auth/2fa-verify-login-otp</code> (limit 5 per minute).
5	Backend	<code>verifyAsync(twoFactorToken)</code> : failure → 400 "Session is Expired". Then: the user exists and is ACTIVE .
6	Backend	<code>verifyOtp(userId, LOGIN, otp)</code> : the newest unused, unexpired code → otherwise 400 "Otp expired or invalid"; <code>attempts ≥ 5</code> → 400 "Too many otp attempts"; wrong code → <code>attempts + 1</code> and 400 "Invalid Otp"; correct → <code>verifiedAt = now</code> .
7	Backend	<code>issueTokenPair</code> : from here it is the same as in 7.1, a session is created and the tokens are returned.
8	Frontend	Tokens are saved to <code>localStorage</code> and <code>Redux</code> , then <code>navigate(redirectTo)</code> .

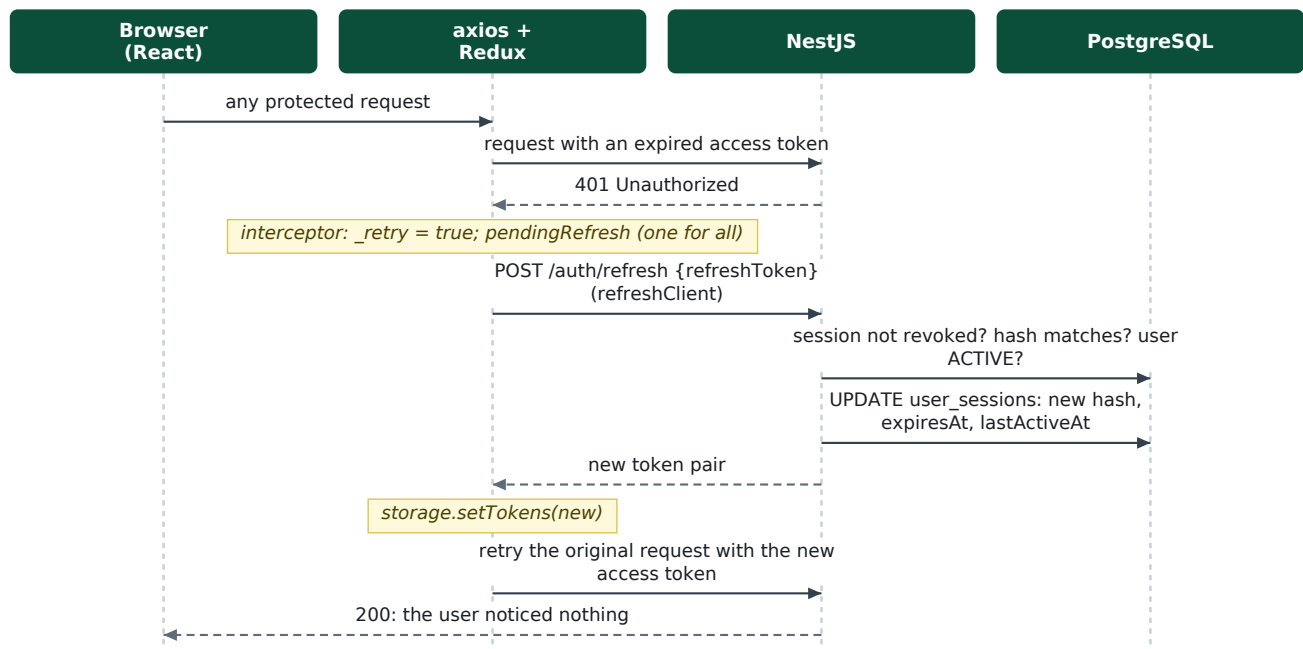
REMEMBER Why both `twoFactorToken` and a code

These are two different proofs. The `twoFactorToken` (signed by the server) proves that "the password was already verified", and the server does not store that state. The **emailed code** proves control of the mailbox. Without the token, codes cannot be tried against someone else's account; without the code, the token is useless.

7.3. Request to a protected endpoint

#	Where	What happens
1	Frontend	The request interceptor takes <code>ACCESS_TOKEN</code> from <code>localStorage</code> and adds <code>Authorization: Bearer <token></code> .
2	Backend	<code>ThrottlerGuard</code> (rate limit) → <code>JwtAuthGuard</code> → Passport strategy <code>jwt</code> : extracts the Bearer token, verifies the signature (<code>JWT_ACCESS_SECRET</code>) and the expiry (<code>ignoreExpiration: false</code>). Failure → 401.
3	Database	<code>JwtStrategy.validate(payload)</code> : looks up <code>user_sessions</code> by <code>payload.sid</code> together with the user.
4	Backend	No session, <code>revokedAt</code> set, or <code>expiresAt</code> in the past → 401 "Session is no longer valid". User not <code>ACTIVE</code> → 401 "This account is not active".
5	Backend	<code>{ userId, email, userType, sessionId }</code> is placed in <code>request.user</code> ; <code>@CurrentUser()</code> hands it to the controller.
6	Backend	<code>ValidationPipe</code> , the controller, the service and Prisma run; the response goes to the client.

7.4. Automatic token refresh



#	Where	What happens
1	Frontend	A request got 401. The interceptor checks the conditions: status 401; the request has not been retried yet (<code>_retry</code>); the request carried an <code>Authorization</code> header (i.e. it is not a login).
2	Frontend	<code>fetchRefreshToken()</code> : if a refresh is already in flight, all requests wait on the same Promise (<code>pendingRefresh</code>). Otherwise <code>requestTokenPair()</code> takes <code>REFRESH_TOKEN</code> and sends <code>POST /auth/refresh</code> through <code>refreshClient</code> .
3	Backend	<code>AuthService.refresh</code> , step 1: <code>verifyAsync</code> with <code>JWT_REFRESH_SECRET</code> (signature and expiry) → otherwise 401 "Invalid refresh token".
4	Backend	Step 2: the session <code>payload.sessionId</code> exists, is not revoked and has not expired.
5	Backend	Step 3: the HMAC of the presented token is compared with <code>refreshTokenHash</code> via <code>timingSafeEqual</code> . Mismatch → 401 (this is how an old, already used refresh token "stops working").
6	Backend	The user must be <code>ACTIVE</code> (otherwise 403).
7	Backend	Step 4 is rotation : <code>signTokenPair</code> creates a new pair, and in the same session <code>refreshTokenHash</code> , <code>expiresAt</code> , <code>lastActiveAt</code> , IP, OS and browser are updated.

#	Where	What happens
8	Frontend	<code>storage.setTokens(new)</code> , and the original request is retried with the new access token. If the refresh fails, <code>storage.clearStorage()</code> and <code>window.location.href = '/auth/login'</code> .

REMEMBER The session is "sliding"

Every successful refresh extends `expiresAt` by another refresh-token lifetime (30 days). If a user signs in regularly, the session never expires; if the user is absent for 30 days, a new login is required.

7.5. Logout and session management

#	Where	What happens
1	Frontend	The logout button in <code>AppSidebar</code> → <code>dispatch(fetchLogout())</code> → <code>POST /auth/logout</code> (Bearer).
2	Backend	<code>AuthService.logout(userId, sessionId)</code> : the session must belong to the user (otherwise 400), then <code>updateMany({ id, revokedAt: null }, { revokedAt: now })</code> .
3	Backend	From this moment <code>JwtStrategy</code> rejects the access token of this session (which could still have up to 15 minutes to live), and the refresh token fails the session check.
4	Frontend	In both <code>fulfilled</code> and <code>rejected</code> , the reducer clears Redux and calls <code>storage.clearStorage()</code> , so the logout happens locally even if the server is unreachable.
5	Frontend	Device list: <code>SessionsList</code> on mount → <code>GET /auth/sessions</code> → the server returns sessions with <code>revokedAt = null</code> , newest first, with an <code>isCurrent</code> flag (the session from the token).
6	Backend	<code>POST /auth/sessions/:id/revoke</code> revokes one session (owner check: someone else's session cannot be revoked). <code>POST /auth/sessions/revoke-others</code> revokes all except the current one (NOT: <code>{ id: currentSessionId }</code>).
7	Frontend	After a revoke the UI requests the session list again.

7.6. Enabling and disabling 2FA

#	Where	What happens
1	Frontend	<code>TwoFactorAuthentication</code> (dialog): the "Send OTP" button → <code>POST /auth/2fa-generate-otp</code> (Bearer, limit 5/min).
2	Backend	<code>twoFactorEnable(userId)</code> with purpose <code>ENABLE_TOGGLE</code> : code, bcrypt hash, 5-minute TTL, invalidation of earlier codes for this purpose, email.
3	Frontend	The dialog switches to a 6-digit input → <code>POST /auth/2fa-verify-enable { otp }</code> (limit 5/min).
4	Backend	<code>verifyOtp(ENABLE_TOGGLE)</code> , then <code>twoFactorEnabled: !user.twoFactorEnabled</code> ; this is a toggle, not an "enable". Response <code>{ success, message: 'Two-factor enabled disabled', data: { twoFactorEnabled } }</code> .
5	Frontend	The reducer updates <code>state.user.twoFactorEnabled</code> , and the Enabled/Disabled badge changes.

REMEMBER Why the `purpose` field exists

The login code (`LOGIN`) and the 2FA-toggle code (`ENABLE_TOGGLE`) are stored in one table but differ by purpose. `verifyOtp` looks for a code **of its own purpose only**, so the code from a "confirm sign-in" email cannot be used to disable 2FA.

7.7. Changing the password (current one is known)

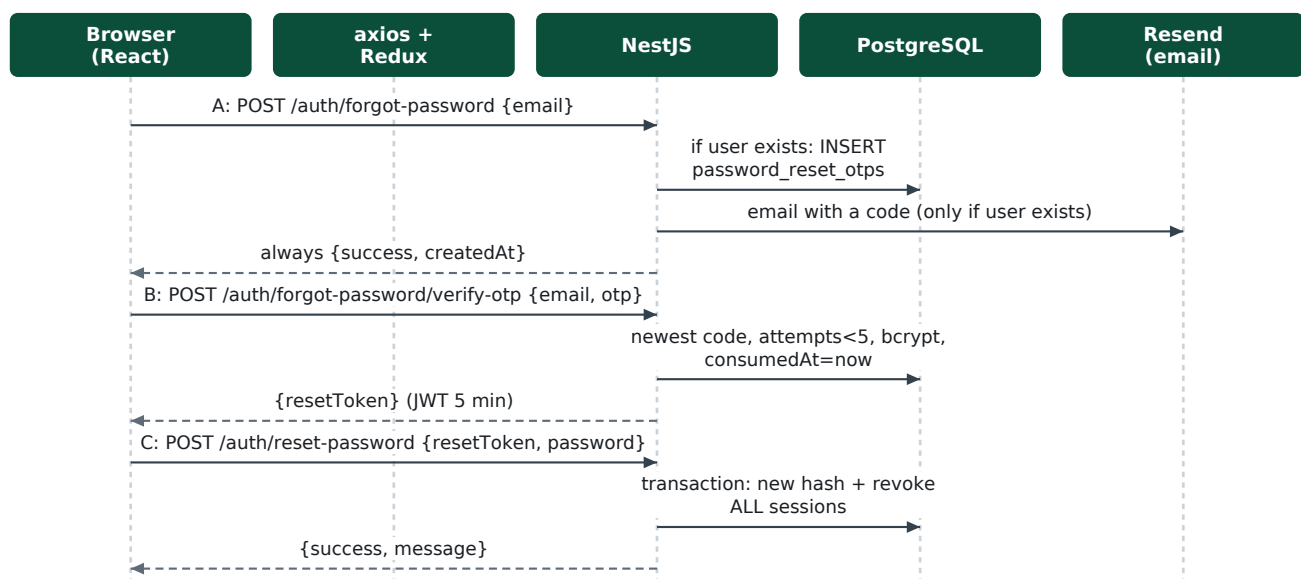
#	Where	What happens
1	Frontend	<code>ChangePassword</code> dialog: current / new / confirmation fields. A mismatch between the new passwords is caught on the client. The button is disabled until everything is filled in.
2	Frontend	<code>PUT /users/change-password { currentPassword, password }</code> (Bearer).
3	Backend	<code>ChangePasswordDto</code> : the new password is 8-72 characters.
4	Backend	<code>UserService.changePassword: bcrypt.compare(currentPassword, hash) → wrong → 401 "Current password is incorrect".</code>
5	Database	In one transaction : the new <code>passwordHash</code> + <code>passwordUpdatedAt</code> , and <code>revokedAt = now</code> for all other sessions of the user (except the current one, <code>id: { not: currentSessionId }</code>).
6	Frontend	The response updates <code>state.user</code> ; on the account page the "changed ... ago" label changes.

REMEMBER Changing the password signs out the other devices

If a password was stolen and the attacker is already signed in, changing the password immediately revokes that session: the attacker's access token stops passing `JwtStrategy`, and the refresh token fails the session check.

7.8. Password recovery (forgotten password)

Three steps, three "proofs": the **emailed code** (control of the mailbox) → the `resetToken` (code passed) → the **new password**.



#	Where	What happens
1	Frontend	<code>ForgotPasswordPage</code> : email input → <code>fetchForgotPassword</code> .
2	Backend	<code>forgotPassword(email)</code> : if the user exists, a 6-digit code, bcrypt hash, 5-minute TTL, email. The response is always <code>{ success: true, message, data: { createdAt } }</code> (for a non-existent email <code>createdAt</code> is simply "now"). This way it is impossible to learn which emails are registered.
3	Frontend	Navigation to <code>/auth/forgot-password/otp</code> with <code>state: { email, createdAt }</code> . The <code>useOtpCountdown</code> hook counts down the remaining 5 minutes from <code>createdAt</code> ; when it runs out, "Resend Otp" is shown.
4	Backend	<code>forgotPasswordOtpVerification(email, otp)</code> : the user (otherwise 400 "Otp does not match"); the newest unused, unexpired code; <code>attempts ≥ 5 → 400</code> ; wrong → <code>attempts + 1</code> ; correct → <code>consumedAt = now</code> (the code is single-use).

#	Where	What happens
5	Backend	A <code>resetToken</code> is signed (<code>JWT_RESET_SECRET</code> , 5 minutes, <code>{ userId }</code>) and returned to the client.
6	Frontend	Navigation to <code>/auth/reset-password</code> (<code>replace: true</code>) with <code>state: { email, resetToken }</code> .
7	Frontend	<code>ResetPasswordPage</code> : check that the two passwords match → <code>POST /auth/reset-password { resetToken, password }</code> .
8	Backend	<code>resetPassword</code> : verifies the <code>resetToken</code> , then <code>UserService.resetForgottenPassword</code> : hashes the new password and, in a transaction, revokes all sessions (the user who forgot the password has no current session).
9	Frontend	The "Password Updated" screen and a "Back to sign in" button. Tokens are not issued ; after a reset the user must sign in again.

7.9. Editing the profile

#	Where	What happens
1	Frontend	<code>EditProfile</code> (dialog): a form with <code>fullName</code> , <code>email</code> , <code>phone</code> , <code>profileImage</code> ; the button is enabled only if something changed.
2	Frontend	<code>PUT /users/me</code> (Bearer) with these fields.
3	Backend	<code>UpdateUserDto</code> + <code>whitelist</code> : extra fields such as <code>userType</code> or <code>status</code> are rejected.
4	Backend	<code>UserService.update</code> → Prisma <code>update</code> with <code>omit</code> . An email already taken → Prisma <code>P2002</code> → 409 "Email is already in use".
5	Frontend	The <code>updateUser.fulfilled</code> reducer updates <code>state.user</code> and recomputes the avatar initials (<code>avatarName</code>).

8. Strengths of the current design

These are the practices the team should understand and be able to explain; they are already in place and should be preserved when the code changes.

Practice	Where	Why it helps
Passwords hashed with bcrypt (cost 10)	<code>login</code> , <code>UserService</code> , <code>seed.ts</code>	A DB leak does not expose passwords
Dummy hash for a non-existent user	<code>ABSENT_USER_PASSWORD_HASH</code>	Response time does not reveal whether the email exists
Identical message "Invalid email or password"	<code>login</code>	No user enumeration
Identical <code>forgot-password</code> response for any email	<code>forgotPassword</code>	The same protection for password reset
Refresh stored in the DB as an HMAC, not the token itself	<code>hashRefreshToken</code>	A DB leak does not yield working tokens
Constant-time token comparison	<code>timingSafeEqual</code>	Protection against timing attacks
Refresh rotation on every renewal	<code>refresh</code>	A stolen old refresh token stops working
Session and status checked on every request	<code>JwtStrategy.validate</code>	Instant logout and blocking
Separate secrets for 4 kinds of tokens	<code>.env</code> , <code>signTokenPair</code>	A token of one kind cannot be passed off as another

Practice	Where	Why it helps
OTP: <code>randomInt</code> , bcrypt hash, 5-min TTL, 5 attempts	<code>twoFactorEnable</code> , <code>verify0tp</code>	Codes cannot be guessed or reused
<code>purpose</code> field on 2FA codes	<code>two_factor_otps</code>	Codes from different flows do not mix
Rate limiting on sensitive routes	<code>@Throttle</code>	Makes brute-forcing passwords and codes harder
Transaction on password change and session revocation	<code>\$transaction</code>	Password and sessions change atomically
<code>ValidationPipe</code> (whitelist) + <code>omit</code>	<code>main.ts</code> , Prisma	No mass assignment, secrets do not reach the client
<code>.env</code> validated at startup, secrets ≥ 32 characters	<code>env.validation.ts</code>	Cannot start with an empty or weak secret
helmet, CORS by origin list	<code>main.ts</code>	Baseline protection of headers and cross-site requests
Single-flight refresh and <code>_retry</code>	<code>lib/axios.ts</code>	No races and no infinite loops
Safe <code>redirect_uri</code>	<code>isSafeRedirectPath</code>	Protection against open redirect
Uniform error format without stack traces	<code>AllExceptionsFilter</code>	The client does not see server internals

9. Known gaps and recommendations

This is a backlog for the team: the normal list of improvements for any project at this stage. Every item was verified against the code; each gives the file, the substance and a proposed fix. Items are ordered by importance.

WARNING 1. No role-based authorization (`userType` is not checked)

Where: `jwt.strategy.ts`, `routes/index.tsx`, `authSlice.ts`

What the code does: The backend puts `userType` into `request.user`, but there is no `RolesGuard/@Roles(...)` anywhere. The frontend does not check the role either: after a successful login any user lands on `/dashboard`. In `authSlice`, `userType` is hard-coded to `"PLATFORM_ADMIN"` everywhere regardless of the response, and the `localStorage` value (`USER_TYPE`) is never read for access decisions. In other words, **login is implemented, but permission separation is not**: for example, a `CUSTOMER` user with a valid password can open the admin console. The practical risk is small today: the other dashboard pages are stubs, and there are no admin endpoints on the backend yet. But as soon as they appear, the missing `RolesGuard` becomes a hole.

Proposed fix: Add `@Roles('PLATFORM_ADMIN') + RolesGuard` (reads `request.user.userType`) on the backend for all admin routes; on the frontend, a `ProtectedRoute` that checks `user.userType`. Hiding buttons on the frontend is not enough: real protection exists only on the server.

IDEA 2. Tokens are stored in `localStorage`

Where: `lib/storage.ts`

What the code does: Any JavaScript on the page (for example injected through XSS or a dependency) can read both tokens, including the 30-day refresh token.

Proposed fix: For production, the refresh token is often kept in a cookie with the `HttpOnly; Secure; SameSite` flags and the access token in memory. This reworks both the backend (issuing the cookie; `credentials: true` is already enabled in CORS) and the frontend. At the current stage `localStorage` is an acceptable trade-off; what matters is that the team understands it.

WARNING 3. Password reset: weak points of the last step**Where:** `auth.service.ts` → `resetPassword`, `dto/reset-password.dto.ts`

What the code does: (a) `resetToken` is a 5-minute JWT and it is **not single-use**: it can be reused until it expires. (b) `verifyAsync` is not wrapped in `try/catch`: for an expired or malformed token a jwt error is thrown that is not an `HttpException`, and the client gets a 500 "Internal server error" instead of 400/401; the `if (!decode)` check after it is unreachable. (c) `ResetPasswordDto` only checks "non-empty string", unlike `ChangePasswordDto` (8-72 characters); `ResetForgottenPasswordDto` with the 8-72 rules is used only as a type in the service, not as the request DTO in the controller.

Proposed fix: Wrap `verifyAsync` in `try/catch` with a `BadRequestException`; add `@MinLength(8)` `@MaxLength(72)` to `ResetPasswordDto`; optionally make the token single-use (store `jti`/a used flag in the DB).

WARNING 4. The 2FA login page watches the wrong loading flag**Where:** `Verify2Fa0tpPage.tsx` and `authSlice.ts`

What the code does: The page reads `state.auth.isTwoFactorVerify0tpLoading` (the flag of the 2FA enable dialog), while the login request sets `isVerify2FaLogin0tpLoading`. As a result, the button and the input on this page are not disabled while the request is in flight.

Proposed fix: Replace the selector with `isVerify2FaLogin0tpLoading`. It is minor, but a good example of why flags should be named unambiguously.

WARNING 5. The otp parameter is optional in the DTO of recovery step 2**Where:** `login-dto.ts` → `ForgotPasswordDto`, `auth.controller.ts`

What the code does: One DTO serves both step 1 (email only) and step 2 (email + otp), so `otp` is marked `@IsOptional` and the controller uses `dto.otp!`. A request without `otp` passes validation and reaches `bcrypt.compare(undefined, ...)`, which most likely ends in a 500.

Proposed fix: Split into two DTOs (`ForgotPasswordDto` and `VerifyForgotPassword0tpDto` with a required `otp`).

IDEA 6. No refresh-token reuse detection**Where:** `auth.service.ts` → `refresh`

What the code does: Rotation exists: the old refresh token no longer matches (the hash differs) and the request simply gets a 401. But the session is not revoked, so if a token was stolen and used before its owner did, the system will not find out.

Proposed fix: The classic scheme: when an **already used** refresh token is presented, revoke the whole session (or the session family) and treat it as a sign of compromise.

WARNING 7. Seed creates an administrator with a simple fixed password**Where:** `backend/prisma/seed.ts`

What the code does: For `PLATFORM_ADMIN` the email and a very simple 8-digit password are hard-coded; on a repeated seed run it is **overwritten** (`update: { passwordHash }`).

Proposed fix: Take the password from an environment variable, do not run seed against a production database without changing the password, and enable 2FA on such an account.

IDEA 8. Minor findings**Where:** various files

What the code does: • `lastActiveAt` is updated only on refresh (about every 15 minutes), not on every request, so in the device list "activity" will be approximate. • `listSessions` does not filter by `expiresAt`, so expired but not revoked sessions may appear in the list. • Changing the email in `PUT /users/me` requires neither a password nor mailbox confirmation. • In the email HTML the user name is inserted without escaping (`Hello ${userName}`). • The empty `create-auth.dto.ts` and `update-auth.dto.ts`, the `twoFactorSecret` field and the `Role` type on the frontend are not used yet. • Typos in names (`clearStoradge`, `ForgotPassowordPage`, `commision-payouts`) are harmless, but are best fixed while the codebase is small.

Proposed fix: Work through these as the project grows.

9.1. What true multi-tenancy requires

- A **Tenant** table (store/organization) and a link between user and tenant (including many-to-many, if one person works in several stores).
- **tenantId** in the session and/or in the access token, and also in **every** domain table (products, orders...).
- A server-side filter by **tenantId** in all queries (for example via a Prisma extension or a base repository), so that forgetting **where: { tenantId }** is impossible.
- Roles inside a tenant (owner, manager...) and **RolesGuard/PermissionsGuard**; the platform admin role is kept separate.
- Tests for cross-tenant leaks: a user of tenant A must not see or modify the data of tenant B.

10. Checklist and review questions

10.1. Checklist: 15 key points

1. **Authentication ≠ authorization.** Login (who the user is) is implemented; permissions (what the user may do) are not yet.
2. The access token is short-lived (15 min) and sent with every request. The refresh token is long-lived (30 days) and used only for renewal.
3. A JWT is **signed but not encrypted**: the payload is readable by anyone, so secrets must not be stored in it.
4. Every session has a row in **user_sessions**; the access token contains **sid**, a reference to it.
5. **JwtStrategy.validate** checks the session in the DB on every request, which is why logout, password change and blocking take effect immediately.
6. The DB holds the **HMAC hash** of the refresh token, not the token itself; comparison uses **timingSafeEqual**.
7. Refresh is **rotated**: after a renewal the old refresh token no longer works.
8. On the frontend there is one refresh for all parallel requests (**pendingRefresh**); otherwise a race breaks the login.
9. **refreshClient** is separate from **Axios** so that the refresh does not loop on itself.
10. 2FA uses two "proofs": **twoFactorToken** (password verified) + the emailed code (control of the mailbox).
11. Codes: 6 digits, bcrypt hash, 5 minutes, 5 attempts, single-use; **purpose** separates the flows.
12. **login** and **forgot-password** do not reveal whether an email exists; **login** also equalizes response time.
13. Password change/reset revokes sessions (all others / all).
14. The frontend makes no security decisions: any check in the browser is only a convenience, real protection is on the server.
15. Tokens in **localStorage** are convenient but vulnerable to XSS; roles are not checked; the reset token is not single-use. These are the main backlog items.

10.2. Onboarding and review questions

Useful for onboarding new team members and for code review of authentication changes: try answering aloud first, then compare with the answers.

Question	Answer
1. Why two tokens rather than one long-lived token?	The access token goes out with every request and therefore lives briefly, so a stolen one expires quickly. The refresh token is used rarely and only on <code>/auth/refresh</code> , so it can live long and is still revocable through its session.
2. How can an access token be "revoked" if a JWT is stateless by itself?	The token contains <code>sid</code> . On every request <code>JwtStrategy</code> reads the session from the DB; if <code>revokedAt</code> is set or the session has expired, the result is 401. This adds a DB query but gives instant revocation.
3. What is stored in <code>user_sessions.refreshTokenHash</code> and why not the token itself?	An HMAC-SHA256 of the token with a server secret. If the DB leaks, a working refresh token cannot be derived from the hash.
4. What happens if two requests get a 401 at the same time?	Thanks to <code>pendingRefresh</code> , only one <code>/auth/refresh</code> goes to the server; both requests wait for the new access token and are retried. Without it, the second refresh with an already used token would fail and sign the user out.
5. Why does <code>login</code> call <code>bcrypt.compare</code> even for a non-existent email?	So that response time is the same and it is impossible to tell by speed whether an email is registered.
6. How does <code>twoFactorToken</code> differ from the emailed code?	The token is a server-signed "password verified, waiting for the second factor"; the code is proof of access to the mailbox. Both are needed.
7. Why do 2FA codes have a <code>purpose</code> field?	So that a login code cannot be used to enable/disable 2FA, and vice versa.
8. Why does <code>forgot-password</code> always answer "success"?	So that it is impossible to find out by brute force which emails exist in the system.
9. What happens to other devices on a password change? And on a reset?	Change: all sessions except the current one are revoked. Reset: all sessions are revoked.
10. What do <code>whitelist: true</code> + <code>forbidNonWhitelisted: true</code> do and what do they protect against?	They strip/forbid fields not described in the DTO. This protects against mass assignment: a client cannot send <code>userType</code> or <code>status</code> in <code>PUT /users/me</code> .
11. Why does <code>isSafeRedirectPath</code> forbid paths starting with <code>//</code>?	The browser treats <code>//evil.com</code> as the address of another site; without the check, <code>?redirect_uri=</code> would let an attacker send a user to a foreign page after login (open redirect).
12. Name the three main weak points of the current authorization.	No role checks (<code>userType</code>); tokens in <code>localStorage</code> ; password reset: the reset token is not single-use, there is no <code>try/catch</code> and no password length check.
13. What is missing from the authentication layer for multi-tenancy?	A tenant entity, <code>tenantId</code> in the session/token and in the data, server-side filtering by tenant, roles inside a tenant and guards.

Appendices

A. How to observe the flows

- **DevTools → Network.** Open the tab, sign in to the app and look at the `login`, `me`, `refresh` requests: Payload shows what the browser sent, Preview shows what the server answered.
- **DevTools → Application → Local Storage.** Find `ACCESS_TOKEN`, `REFRESH_TOKEN`, `USER_TYPE`. To decode the payload: `JSON.parse(atob(token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/')))` in the console.
- **Check the automatic refresh.** Set `JWT_ACCESS_EXPIRES_IN="1m"` in `backend/.env`, sign in and after a minute perform any action: Network shows `401` → `/auth/refresh` → the request repeated with `200`.
- **Check revocation.** Sign in from two browsers, revoke the second session in "Active sessions"; in the second browser the next request returns `401` and the refresh fails.
- **Prisma Studio** (`npm run prisma:studio` in `backend/`): watch rows appear in `user_sessions` and `two_factor_otp`s and how `revokedAt`, `attempts`, `verifiedAt` get filled in.

Running the project

```
# backend/
cp .env.example .env          # fill in DATABASE_URL, 4 secrets (>=32 characters), RESEND_*
npm install
npm run prisma:deploy          # apply migrations (or prisma:migrate in development)
npm run db:seed               # create the administrator (see item 7 in part 9)
npm run start:dev             # http://localhost:4000

# platform-admin/
npm install
npm run dev                   # http://localhost:3002 (VITE_APP_API_BASE_URL, defaults to localhost:4000)
```

B. Authentication file map

File	Purpose
<code>backend/prisma/schema.prisma</code>	Models and enums
<code>backend/src/main.ts</code>	helmet, ValidationPipe, error filter, CORS
<code>backend/src/app.module.ts</code>	Config, Throttler, Passport, modules
<code>backend/src/config/env.validation.ts</code>	<code>.env</code> validation at startup
<code>backend/src/auth/auth.controller.ts</code>	<code>/auth/*</code> endpoints
<code>backend/src/auth/auth.service.ts</code>	All logic for login, tokens, sessions, 2FA, password reset
<code>backend/src/auth/strategies/jwt.strategy.ts</code>	Access-token and session verification
<code>backend/src/auth/dto/*, types/*</code>	DTOs and token/response types
<code>backend/src/common/guards/jwt-auth.guard.ts</code>	Guard for protected routes
<code>backend/src/common/decorators/*</code>	<code>@CurrentUser()</code> , <code>@ParseUserAgent()</code>
<code>backend/src/common/filters/all-exceptions.filter.ts</code>	Uniform error format
<code>backend/src/users/*</code>	Profile, password change, password reset
<code>backend/src/email/email.service.ts</code>	Emails with codes (Resend)
<code>backend/prisma/seed.ts</code>	Administrator creation
<code>platform-admin/src/lib/axios.ts</code>	Interceptors and automatic token refresh
<code>platform-admin/src/lib/storage.ts</code>	localStorage
<code>platform-admin/src/api/auth.ts</code>	API calls
<code>platform-admin/src/store/auth/authSlice.ts</code>	Redux authentication logic
<code>platform-admin/src/routes/index.tsx</code> , <code>components/ProtectedRoute.tsx</code> , <code>layouts/AuthLayout.tsx</code>	Routes and page protection
<code>platform-admin/src/pages/auth/*</code>	Login, 2FA and password recovery screens
<code>platform-admin/src/pages/dashboard/account/*</code>	Profile, password, 2FA, sessions

C. For more detail

The repository already contains earlier materials: [AUTH_FLOW.md](#) (a walkthrough with diagrams and "On the wire" JSON examples) and [AUTH_FLOW_RU.pdf](#) (a Russian walkthrough with screenshots of the Network tab). These notes were rebuilt from the current code (commit [ee4653a](#)) as team documentation: a glossary of concepts, a layer-by-layer breakdown, flows, a backlog of known gaps and review questions.